



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Lechuga Castillo Shareny Ixchel

N° de Cuenta: 319004252

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

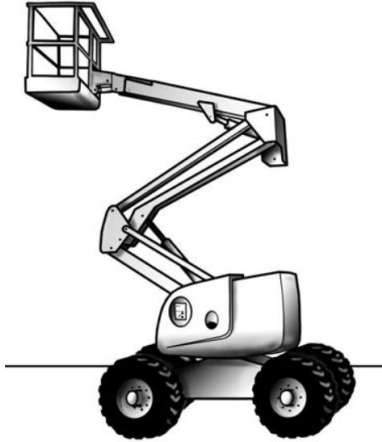
FECHA DE ENTREGA LÍMITE: 18-09-2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Terminar la Grúa con:

- cuerpo(prisma rectangular)
- brazo
- base (pirámide cuadrangular)
- 4 llantas(4 cilindros) con teclado se pueden girar las 4 llantas por separado



El código que obtenemos es el siguiente:

```
356 //----- EJERCICIO 1 -----
357
358
359 //----- BASE -----
360 model = glm::mat4(1.0);
361 model = glm::translate(model, glm::vec3(0.0f, 2.5f, -4.0f));
362 modelaux = model;
363 model = glm::scale(model, glm::vec3(4.0f, 2.0f, 2.0f));
364 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
365 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
366 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
367 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
368 color = glm::vec3(1.0f, 0.0f, 0.0f); // rojo
369 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); // para cambiar el color del objetos
370 meshList[4] -> RenderMesh();
371
372 //----- LLANTA 1-----
373 model = modelaux;
374 model = glm::translate(model, glm::vec3(-1.9f, -1.7f, -1.0f));
375 model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
376 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
377 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
378 color = glm::vec3(0.0f, 0.0f, 0.0f); // rojo
379 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); // para cambiar el color del objetos
380 sp.render();
381
382 //----- LLANTA 2-----
383 model = modelaux;
384 model = glm::translate(model, glm::vec3(1.9f, -1.7f, -1.0f));
385 model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
386 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, 1.0f));
387 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
388 color = glm::vec3(0.0f, 0.0f, 0.0f); // rojo
389 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); // para cambiar el color del objetos
390 sp.render();
```

```

392 //----- LLANTA 3-----
393 //Letra M para rotación
394 model = modelaux;
395 model = glm::translate(model, glm::vec3(-1.9f, -1.7f, 1.0f));
396 model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
397 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
398 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
399 color = glm::vec3(0.0f, 0.0f, 0.0f); // rojo
400 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); // para cambiar el color del objetos
401 sp.render();
402
403 //----- LLANTA 4-----
404 //Letra N para rotación
405 model = modelaux;
406 model = glm::translate(model, glm::vec3(1.9f, -1.7f, 1.0f));
407 model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
408 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
409 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
410 color = glm::vec3(0.0f, 0.0f, 0.0f); // rojo
411 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); // para cambiar el color del objetos
412 sp.render();
413
414 //CREANDO LA CABINA DE LA GRÚA
415 //para reiniciar la matriz de modelo con valor de la matriz identidad
416 model = glm::mat4(1.0);
417 model = glm::translate(model, glm::vec3(0.0f, 4.0f, -4.0f));
418 modelaux = model;
419 model = glm::scale(model, glm::vec3(4.0f, 2.0f, 2.0f));
420 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
421 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
422 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
423 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
424 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
425 color = glm::vec3(1.0f, 0.0f, 0.0f); // rojo
426 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); // para cambiar el color del objetos
427 meshList[0]->RenderMesh();
428 model = modelaux;

```



```

430 //ARTICULACION CABINA-BRAZO 1
431 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
432 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
433
434 sp.render();
435 shaderList[0].useShader();
436 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // seguro
437 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
438
439 //-----Creando el brazo de una grúa-----
440 //articulacion1 hasta articulación5 sólo son puntos de rotación o articulación, en este caso no di
441
442 //primer brazo que conecta con la cabina
443 model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
444 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
445 modelaux = model;
446 model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
447
448 //rotación alrededor de la articulación que une con la cabina
449 //Traslación inicial para posicionar en -Z a los objetos
450 //model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
451 //otras transformaciones para el objeto
452 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
453 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
454 //se programe cambio entre proyección ortogonal y perspectiva
455 color = glm::vec3(1.0f, 1.0f, 0.0f);
456 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
457 meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
458 //meshList[3]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cu
459 //sp.render(); //dibuja esfera
460 model = modelaux;
461 //SEGUNDA ARTICULACIÓN
462 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
463 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
464 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
465 sp.render();

```

```

467 //-----segundo brazo-----
468
469 //Comentar y se modifica para agregar la jerarquia:
470 //usar una matriz temporal o auxiliar
471
472 //model = glm::mat4(1.0);
473 model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
474
475 //Traslación inicial para posicionar en -Z a los objetos
476 //model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
477 //otras transformaciones para el objeto
478 modelaux = model;
479 model = glm::scale(model, glm::vec3(1.0f, 5.0f, 1.0f));
480 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
481 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
482 //se programe cambio entre proyección ortogonal y perspectiva
483 color = glm::vec3(1.0f, 1.0f, 0.0f);
484 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
485 meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
486
487 //ARTICULACION3
488 model = modelaux;
489 model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
490 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
491 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
492 sp.render();
493
494 //-----Tercer brazo-----
495 model = glm::translate(model, glm::vec3(0.0f, -3.2f, 0.0f));
496
497 //Traslación inicial para posicionar en -Z a los objetos
498 //model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
499 //otras transformaciones para el objeto
500 modelaux = model;
501 model = glm::scale(model, glm::vec3(1.0f, 5.0f, 1.0f));
502 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
503
504
505 color = glm::vec3(1.0f, 1.0f, 0.0f);
506 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
507 meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
508 //-----ARTICULACION 4-----
509 model = modelaux;
510 model = glm::translate(model, glm::vec3(0.0f, -3.2f, 0.0f));
511 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
512 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
513 sp.render();
514 shaderList[0].useShader();
515 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // seguro
516 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
517
518 //-----Canasta-----
519 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
520 modelaux = model;
521 model = glm::scale(model, glm::vec3(2.5f, 2.5f, 2.5f));
522 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
523 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
524 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
525 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
526 color = glm::vec3(1.0f, 0.0f, 0.0f); // rojo
527 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); // //para cambiar el color del objetos
528 meshList[0]->RenderMesh();
529 model = modelaux;
530
531 shaderList[0].useShader();
532 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // seguro
533 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));

```

El ejecutable que obtenemos es el siguiente:



2.- Crear un animal robot 3d

-Instanciando cubos, pirámides, cilindros, conos, esferas:

-4 patas articuladas en 2 partes (con teclado se puede mover las dos articulaciones de cada pata por separado)

-cola articulada o 2 orejas articuladas. (con teclado se puede mover la cola o cada oreja independiente)

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Hicimos un perrito en este caso, el código que obtenemos es:

```
536 //----- EJERCICIO 2 -----
537
538 model = glm::mat4(1.0);
539 //----- Cuerpo
540 model = glm::translate(model, glm::vec3(0.0f, 6.0f, -4.0f));
541 modelaux = model;
542 model = glm::scale(model, glm::vec3(1.0f, 2.0f, 1.0f)); //Se estableció 5
543 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
544 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
545 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
546 color = glm::vec3(1.0f, 0.47f, 0.13f);
547 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
548 //meshList[0]->RenderMesh(); //dibuja cubo, pirámide triangular, pirámide base cuadrangular
549 meshList[2]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro y cono
550 model = modelaux;
551 //----- Collar
552 model = glm::translate(model, glm::vec3(0.0f, 1.05f, 0.0f));
553 modelaux = model;
554 model = glm::scale(model, glm::vec3(1.1f, 0.1f, 1.1f));
555 color = glm::vec3(1.0f, 0.0f, 0.0f);
556 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
557 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
558 //meshList[0]->RenderMesh();
559 meshList[2]->RenderMeshGeometry();
560 model = modelaux;
561
562 //----- Cascabel
563 model = glm::translate(model, glm::vec3(0.0f, -0.27f, 1.2f));
564 modelaux = model;
565 model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
566 color = glm::vec3(1.0f, 0.84f, 0.0f);
567 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
568 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
569 sp.render();
570 model = modelaux;
```

```

571 //----- Cola
572 //-----Articulacion 1
573 model = glm::translate(model, glm::vec3(0.0f, -1.2f, -2.3f));
574 //TECLA F
575 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
576 //dibujar una pequeña esfera
577 //model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
578 //glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
579 //sp.render();
580 model = glm::translate(model, glm::vec3(0.0f, 1.0f, -0.4f));
581 model = glm::rotate(model, glm::radians(-20.0f), glm::vec3(1.0f, 0.0f, 0.0f));
582 model = glm::scale(model, glm::vec3(0.5f, 3.0f, 0.5f));
583 color = glm::vec3(1.0f, 0.47f, 0.13f);
584 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
585 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
586 meshList[0]->RenderMesh();
587 model = modelaux;
588 //----- Cabeza
589 model = glm::translate(model, glm::vec3(0.0f, 1.17f, -1.2f));
590 modelaux = model;
591 model = glm::scale(model, glm::vec3(2.2f, 1.7f, 2.2f));
592 color = glm::vec3(1.0f, 0.47f, 0.13f);
593 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
594 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
595 meshList[0]->RenderMesh();
596 modelaux = model;
597 //-----Ojo izquierdo
598 model = glm::translate(model, glm::vec3(-0.2f, 0.2f, 0.45f));
599 modelaux = model;
600 model = glm::scale(model, glm::vec3(0.1f, 0.2f, 0.2f));
601 modelaux = model;
602 color = glm::vec3(0.0f, 0.0f, 0.0f);
603 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
604 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
605 meshList[0]->RenderMesh();
606 modelaux = model;

```

```

607 //-----Ojo derecho
608 model = glm::translate(model, glm::vec3(4.3f, 0.04f, 0.05f));
609 modelaux = model;
610 //model = glm::scale(model, glm::vec3(0.5f, 0.6f, 0.6f));
611 color = glm::vec3(0.0f, 0.0f, 0.0f);
612 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
613 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
614 meshList[0]->RenderMesh();
615 model = modelaux;
616 //-----Nariz
617 model = glm::translate(model, glm::vec3(-2.3f, -1.27f, 0.3f));
618 modelaux = model;
619 model = glm::scale(model, glm::vec3(1.0f, 0.5f, 0.5f));
620 color = glm::vec3(1.0f, 0.75f, 0.8f);
621 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
622 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
623 sp.render();
624 model = modelaux;
625 //-----Boca
626 model = glm::translate(model, glm::vec3(0.0f, -0.8f, -0.2f));
627 modelaux = model;
628 model = glm::scale(model, glm::vec3(0.2f, 0.6f, 0.6f));
629 color = glm::vec3(0.0f, 0.0f, 0.0f);
630 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
631 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
632 meshList[0]->RenderMesh();
633 model = modelaux;
634
635 model = glm::translate(model, glm::vec3(0.0f, -0.25f, 0.0f));
636 modelaux = model;
637 model = glm::scale(model, glm::vec3(1.5f, 0.1f, 0.6f));
638 color = glm::vec3(0.0f, 0.0f, 0.0f);
639 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
640 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
641 meshList[0]->RenderMesh();
642 model = modelaux;

```



```

643 //-----Oreja izquierda
644 model = glm::translate(model, glm::vec3(-5.1f, 3.8f, -2.5f));
645 modelaux = model;
646 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
647 model = glm::scale(model, glm::vec3(0.9f, 3.8f, 0.5f));
648 color = glm::vec3(1.0f, 0.8f, 0.6f);
649 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
650 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
651 //meshList[0]->RenderMesh();
652 meshList[3]->RenderMeshGeometry();
653 model = modelaux;
654 //-----Oreja derecha
655 model = glm::translate(model, glm::vec3(10.3f, 0.1f, 0.1f));
656 modelaux = model;
657 model = glm::rotate(model, glm::radians(-45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
658 model = glm::scale(model, glm::vec3(0.9f, 3.8f, 0.5f));
659 color = glm::vec3(1.0f, 0.8f, 0.6f);
660 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
661 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
662 //meshList[0]->RenderMesh();
663 meshList[3]->RenderMeshGeometry();
664 model = modelaux;
665
666 //-----Brazo Izquierdo
667 //----- Articulacion 2 (Brazo izquierdo, parte pegada al cuerpo)
668 model = glm::translate(model, glm::vec3(-10.0f, -6.7f, 0.0f));
669 // TECLA H
670 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
671 modelaux = model;
672 // Dibujar parte del brazo izquierdo pegada al cuerpo
673 model = glm::translate(model, glm::vec3(-0.3f, 0.3f, 1.2f));
674 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
675 model = glm::scale(model, glm::vec3(2.0f, 3.0f, 1.0f));
676 color = glm::vec3(1.0f, 0.6f, 0.3f);
677 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
678 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
679 meshList[0]->RenderMesh();
680 model = modelaux;
681
682 //----- Articulacion 3 (Brazo izquierdo, cubo pequeño)
683 model = glm::translate(model, glm::vec3(-0.4f, 0.3f, 2.0f));
684 // TECLA J
685 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
686 modelaux = model;
687 // Dibujar el cubo pequeño del brazo izquierdo
688 model = glm::translate(model, glm::vec3(0.0f, 0.35f, 1.1f));
689 model = glm::scale(model, glm::vec3(2.0f, 1.8f, 0.8f));
690 color = glm::vec3(1.0f, 0.8f, 0.6f);
691 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
692 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
693 meshList[0]->RenderMesh();
694 model = modelaux;
695 model = modelOriginal;
696 //-----BRAZO DERECHO
697 //----- Articulacion 4 (Brazo derecho, parte pegada al cuerpo)
698 model = glm::translate(model, glm::vec3(0.8f, 6.70f, -3.9f));
699 // TECLA K
700 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(1.0f, 0.0f, 0.0f));
701 modelaux = model;
702 // Dibujar parte del brazo derecho pegada al cuerpo
703 model = glm::translate(model, glm::vec3(0.4f, 0.0f, 0.5f)); // Simétrico al brazo izquierdo
704 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
705 model = glm::scale(model, glm::vec3(0.47f, 1.1f, 0.33f));
706 color = glm::vec3(1.0f, 0.6f, 0.3f);
707 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
708 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
709 meshList[0]->RenderMesh();
710 model = modelaux;
711 //----- Articulacion 5 (Brazo derecho, cubo pequeño)
712 model = glm::translate(model, glm::vec3(0.4f, -0.05f, 0.1f));
713 // TECLA L
714 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
715 modelaux = model;
716 // Dibujar el cubo pequeño del brazo derecho
717 model = glm::translate(model, glm::vec3(0.0f, 0.15f, 1.1f));
718 model = glm::scale(model, glm::vec3(0.54f, 0.65f, 0.3f));
719 color = glm::vec3(1.0f, 0.8f, 0.6f);

```

```

719 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
720 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
721 meshList[0]->RenderMesh();
722 //-----Pie Izquierdo
723 model = modelOriginal;
724 //----- Articulacion 6
725 model = glm::translate(model, glm::vec3(-0.8f, 4.44f, -4.3f));
726 // TECLA L
727 model = glm::rotate(model, glm::radians(-mainWindow.getarticulacion6()), glm::vec3(0.0f, 1.0f, 0.0f));
728 modelaux = model;
729 // Dibujar parte del pie izquierdo pegada al cuerpo
730 model = glm::translate(model, glm::vec3(0.1f, 0.3f, 0.7f));
731 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
732 model = glm::scale(model, glm::vec3(0.5f, 1.3f, 0.5f));
733 color = glm::vec3(1.0f, 0.6f, 0.3f);
734 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
735 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
736 meshList[0]->RenderMesh();
737 model = modelaux;
738
739 //----- Articulacion 7
740 model = glm::translate(model, glm::vec3(0.12f, 0.1f, 1.5f));
741 // TECLA M
742 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(1.0f, 0.0f, 0.0f));
743 modelaux = model;
744 // Dibujar el cubo pequeño del pie izquierdo
745 model = glm::translate(model, glm::vec3(0.0f, 0.2f, 0.0f));
746 model = glm::scale(model, glm::vec3(0.5f, 0.6f, 0.3f));
747 color = glm::vec3(1.0f, 0.8f, 0.6f);
748 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
749 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
750 meshList[0]->RenderMesh();
751 model = modelOriginal;

753 //-----Pie Derecho
754 model = modelOriginal;
755 //----- Articulacion 8
756 model = glm::translate(model, glm::vec3(0.6f, 4.44f, -4.3f));
757 model = glm::rotate(model, glm::radians(150.0f), glm::vec3(0.0f, 1.0f, 0.0f));
758 // TECLA N
759 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, -1.0f, 0.0f));
760 modelaux = model;
761 // Dibujar parte del pie derecho pegada al cuerpo
762 model = glm::translate(model, glm::vec3(0.1f, 0.3f, 0.7f));
763 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
764 model = glm::scale(model, glm::vec3(0.5f, 1.3f, 0.5f));
765 color = glm::vec3(1.0f, 0.6f, 0.3f);
766 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
767 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
768 meshList[0]->RenderMesh();
769 model = modelaux;
770 //----- Articulacion 9 (Pie derecho, cubo pequeño)
771 model = glm::translate(model, glm::vec3(0.12f, 0.1f, 1.5f));
772 // TECLA B
773 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(1.0f, 0.0f, 0.0f));
774 modelaux = model;
775 // Dibujar el cubo pequeño del pie derecho
776 model = glm::translate(model, glm::vec3(0.0f, 0.2f, 0.0f));
777 model = glm::scale(model, glm::vec3(0.5f, 0.6f, 0.3f));
778 color = glm::vec3(1.0f, 0.8f, 0.6f);
779 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
780 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
781 meshList[0]->RenderMesh();
782 model = modelOriginal;
783
784 glUseProgram(0);
785 mainWindow.swapBuffers();
786
787 return 0;
788 }

```


El ejecutable que obtenemos es el siguiente:



Problemas:

Ahora si me costó demasiado trabajo esta práctica, desde cómo me sentía de salud hasta enojarme por no poner bien las coordenadas, tuve problemas al rotar e insertar bien cada articulación y cada parte de los dibujos, se me esta complicando entender en que parte debo de poner el model = modelaux; hice muchas pruebas porque no entendía al cien esto.

Conclusión:

- a. Los ejercicios del reporte: Ahora si fue más complicado, más que nada el animal que teníamos que realizar, primero no tenia idea de que hacer y después creo que me complique demasiado, solo por tomar en cuenta un dibujo de un perrito para colorear.
- b. Comentarios generales: En esta ocasión no puedo dar comentarios sobre eso, ya que no pude asistir, pero supongo que han sido como siempre, la explicación es rápida y trata de hacerla lo más clara posible pero ahora si ya empiezan las prácticas más difíciles.
- c. Conclusión: Cada vez esta subiendo la complejidad, y es el claro ejemplo de que no puedes atrasarte ni una sola práctica porque es cada vez más difícil entender todo. Se debe tener claro todo lo que se ve en cada práctica porque todo lo llevamos a cabo.